Compiler Technology Report

Lexical Analysis: Principles, Design and Implementation in Python

Course: Compiler Technology

Student Name: Ameer Subhan Hussain

Student ID: 202324405009

Submission Date: 6th June, 2026

Student Signature: Ameer

1. Introduction

A compiler is a computer program that converts source code written in a high-level language into machine code or intermediate code. It is divided and has several sequential stages: Lexical Analysis, Syntax Analysis, Semantic Analysis, Intermediate Code Generation, Optimization, and Final Code Generation. As the example for this report, the lexical analysis of the first and most basic phase is considered, with a lexer in Python generated throughout this course as an example.

2. Roles and Functions of Compilers

A compiler performs much work more than just a translation. It imposes language rules, detects errors early and generates efficient target code.

Lexer: It is responsible for converting characters to tokens.

Parser: It is responsible to construct a syntax tree. This separation by clean enables each phase to be developed and tested separately.

3. Lexical Analysis: The First Compiler Phase

Lexical analysis is the part of the compiler that builds a lexicon from the characters in the source code, which is the smallest possible meaningful parts of the language. The value of a token is the actual literal of the program (e.g. "while", "x", "42") and the type is the meaning of the token (e.g. KEYWORD, IDENTIFIER, INTEGER). The lexer ignores spaces and counts the line numbers to help with precise error reporting in all the following phases.

3.1 Token Definitions (Lines 1–19)

Using Python regular expressions, 13 token types are defined in the lexer. Note the use of word-boundaries (\b) in keywords to prevent the word interface from being interpreted as the keyword int. The TOKENS list is designed intentionally so that it is possible to use reserved words as identifiers without this causing confusion.

```
1  import re
2  import sys
3
4  # ===== TOKEN DEFINITIONS =====
5  TOKENS = [
6      ('KEYWORD', r'\b(int|float|double|char|void|if|else|while|for|return)\b'),
7      ('IDENTIFIER', r'[a-zA-Z_][a-zA-Z0-9_]*'),
8      ('FLOAT', r'\d+\.\d+'),
9      ('INTEGER', r'\d+'),
10     ('OPERATOR', r'[+\-*/=<>!]+'),
11     ('SEMICOLON', r';'),
12     ('COMMA', r','),
13     ('LPAREN', r'\('),
14     ('RPAREN', r'\)'),
15     ('LBRACE', r'\{'),
16     ('RBRACE', r'\}'),
17     ('WHITESPACE', r'[ \t]+'),
```

Figure 1: lexer.py: imports and TOKENS list (lines 1–17)

3.2 The Core Scanning Loop (Lines 21–62)

The lexer function reads the source text, one character at a time. It will search for every pattern in order for each position, the first match will win. Whitespace is eaten up without error. Newlines increment the line counter. All the other matches are saved as a dictionary with the line number, token type, and raw value in the token list.

```
18      ('NEWLINE', r'\n'),
19  ]
20
21  def lexer(code):
22      tokens = []
23      line_num = 1
24      pos = 0
25
26      print("\n" + "="*60)
27      print("   TOKENS FOUND")
28      print("="*60)
29      print(f"{'Line':<8} {'Token Type':<20} {'Value'}")
30      print("-"*60)
31
32      while pos < len(code):
33          matched = False
34
35          for token_type, pattern in TOKENS:
36              regex = re.compile(pattern)
37              match = regex.match(code, pos)
38
39              if match:
40                  value = match.group(0)
41
42                  if token_type == 'NEWLINE':
43                      line_num += 1
44                  elif token_type != 'WHITESPACE':
45                      print(f"{line_num:<8} {token_type:<20} {value}")
46                      tokens.append({
47                          'line': line_num,
48                          'type': token_type,
49                          'value': value
50                      })
51
52                  pos = match.end()
53                  matched = True
54                  break
55
56          if not matched:
57              print(f"{line_num:<8} {'ERROR':<20} Unknown: '{code[pos]}'")
58              pos += 1
```

Figure 2 : lexer.py: lexer() function core scanning loop (lines 18–58)

3.3 Command-Line Entry Point (Lines 65–81)

The Command-Line Entry Point is located in the region from line 65 to 81 (3.3). The lexer is written as a standalone lexer. It reads a source file from the command line, feeds the file to the lexer, and outputs a nicely-formatted table of all the tokens recognized. This facilitates testing in isolation from the rest of compiler pipeline.

```
59
60      print("-"*60)
61      print(f"Total tokens: {len(tokens)}")
62      return tokens
63
64
65  # ===== MAIN =====
66  if __name__ == "__main__":
67      if len(sys.argv) < 2:
68          print("Usage: python lexer.py <filename>")
69          print("Example: python lexer.py test.txt")
70          sys.exit(1)
71
72      filename = sys.argv[1]
73
74      try:
75          with open(filename, 'r') as file:
76              code = file.read()
77          print(f"\nAnalyzing file: {filename}")
78          tokens = lexer(code)
79          print("\nAnalysis complete!")
80      except FileNotFoundError:
81          print(f"Error: File '{filename}' not found!")
```

Figure 3 : lexer.py: main entry point and file reading (lines 65–81)

4. Classification of Compilers

There are various ways of categorising compilers. Targeting by type: native compilers generate machine code for the host CPU; cross compilers generate machine code for another CPU (more often used for embedded systems); transpilers translate from a high-level language to another (TypeScript to JavaScript); bytecode compilers generate virtual-machine code (Java to JVM). By pass count: Single-pass compilers are quick but are limited; Multi-pass compilers (LLVM, GCC) can process the program several times for more profound optimisation. There are two types of compilers: Ahead-of-Time (AOT) compilers compile ahead of time, while Just-in-Time (JIT) compilers compile on the fly using data collected during runtime (Java JVM, V8).

The Course Lexer is the front end of an educational, one pass compiler. It creates the flat token list in one scan, and then the parser consumes the list in the subsequent scan.

5. Applications of Compiler Technology

Compiler techniques are not just used to create executables. Real-time syntax highlighting and underlining of errors as the programmer types is provided by incremental lexing in IDEs. Static analysis tools like Clang Analyzer and SonarQube construct build CFGs that analyze some of the code without running it - and catch bugs. Each of the domain specific languages (DSLs) SQL, GLSL shader language, hardware description languages, etc. have their own lexer and parser. For instance, compilers like TVM and TensorFlow XLA generate optimized computation

graph for neural networks and optimize them for GPUs by performing various optimizations, including operator fusion.

6. Personal Reflection on Compiler Design

Prior to taking this course, I had an image of the compiler as a black box. Writing the lexer was my first time getting the true meaning of 'parsing a language' at a low-level. The crucial idea was that source code is nothing more than a random sequence of characters until it is interpreted by a program and it is the TOKENS list that contains the key to this meaning. Learning how to write the token definitions to me was such learning about the decisions made when designing a language, and they reside within the lexer. It is done so in just one regex pattern and its index in the list this is the power of regular expressions. It is achieved with a single regular expression and the position of 'int' in the list regular expressions to the rescue!When I implemented line-number tracking, it became apparent to me why it is important to have good error messages: The difference between 'syntax error' and 'syntax error on line 7, near for' is only an integer that is held during scanning. The most memorable lesson was the clarity of the feed through of the compiler phases. The parser never gets access to the raw characters that the lexer generated, but instead receives the structured list of tokens. This separation of concerns has been something I've always done with any software I've written.

7. Conclusion

Lexical analysis converts raw source text into a structured token stream that all subsequent compiler phases depend on. The course lexer demonstrates the core principles: ordered regex matching, whitespace discarding, keyword priority, line-number tracking, and clean error reporting. These same principles underpin production lexers in GCC, Clang, and every other modern compiler. Understanding them provides a solid foundation for understanding how programming languages work at their deepest level.

References

Lattner, C. and Adve, V. (2004) 'LLVM: A compilation framework for lifelong program analysis and transformation', Proceedings of the 2nd IEEE/ACM International Symposium on Code Generation and Optimization (CGO 2004), pp. 75–88. Available at: https://doi.org/10.1109/CGO.2004.1281665.

Aho, A.V., Lam, M.S., Sethi, R. and Ullman, J.D. (2006) *Compilers: Principles, Techniques, and Tools*. 2nd edn. Pearson.

Python Software Foundation (2026) *re - Regular expression operations*. Available at: https://docs.python.org/3/library/re.html

Python Software Foundation (2006) *Lexical analysis*. Available at: https://edoras.sdsu.edu/doc/Python-Docs-2.5/ref/lexical.html

Kang, J., Kim, Y., Song, Y., Lee, J., Park, S., Shin, M.D. and Kim, Y. (2018) 'Crellvm: verified credible compilation for LLVM', *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2018)*. Available at: https://doi.org/10.1145/3192366.3192377